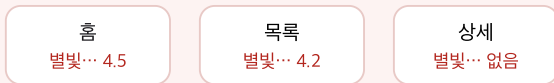


같은 영화가 화면마다 다르면 안 되니까 — Mock 데이터 한곳에

1 화면마다 데이터를 적으면 생기는 일

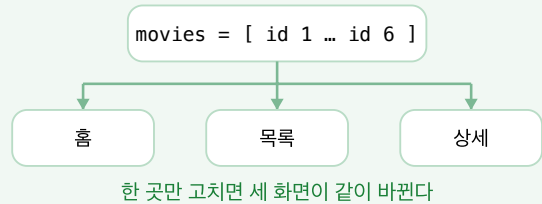
홈·목록·상세는 같은 영화를 보여줘야 한다. 각자 적으면 같은 영화의 제목·평점이 화면마다 달라진다.

✗ 화면마다 하드코딩



같은 영화인데 값이 제각각
하나 고치면 나머지 두 곳도 찾아 고쳐야 한다

✓ mock_movies.dart 한 곳



기억할 것 — 데이터는 한 곳에 두고, 화면은 읽기만 한다.

2 없는 영화 ID로 들어오면?

주소창에 `/movies/999` 를 직접 칠 수도 있다. 그래서 찾기 함수는 `Movie?` (물음표 = 없을 수도 있음)를 돌려 준다.



! 는 "절대 null 아님"이라고 단언하는 표시다. 단언이 틀리면 그 자리에서 앱이 멈춘다. 없을 수 있는 값은 `null`로 받고 화면에서 처리한다.

목록 위젯 세 개는 딱 한 가지씩만 다르다

셋 다 `itemCount` (몇 개)와 `itemBuilder` (하나를 어떻게 그릴지)를 받는다. 여기까지는 완전히 같다.

ListView.builder



한 방향으로 그냥 나열한다.

내 화면: 없음

ListView.separated

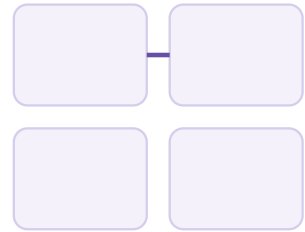


항목 사이에 간격·구분선을 넣는다.

항목 3개 → 사이는 2번

내 화면: 홈 가로 줄(12px), 장르 Chip 줄(8px)

GridView.builder



한 줄에 2개 이상 배치한다.

`gridDelegate` 하나가 더 붙는다

내 화면: 영화 목록 2열

고르는 기준 — 사이에 뭔가 넣고 싶으면 `separated`, 한 줄에 여러 개면 `GridView`, 나머지는 `builder`.

3 왜 `.builder` 를 쓰나

영화가 6개든 600개든, 화면에 보이는 것만 그린다. 한 번에 다 만들지 않아서 스크롤이 끊기지 않는다.

화면(보이는 곳)
카드 4장만 생성

→ 스크롤하면 그때 다음 카드를 만든다 — 600개여도 처음 비용은 같다

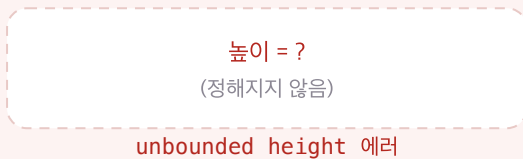
여기서 꼭 한 번은 막힌다

가로 목록 · 카드 비율

4 가로 목록은 높이를 아무도 안 정해 준다

세로 목록은 부모가 높이를 준다. 가로 목록은 "위아래로 얼마나 차지할지"를 아무도 모른다 → 에러가 난다.

✗ Column 안에 바로 가로 ListView



✓ SizedBox로 높이를 정해 준다



카드 너비도 같은 이유로 직접 정한다 — `MovieCard(width: 136)`. 세로 목록에서는 둘 다 신경 쓸 일이 없다.

5 childAspectRatio = 가로 ÷ 세로

칸의 모양(비율)만 정한다. 실제 크기는 화면 너비에 맞춰 따라온다. 값이 작을수록 세로로 길어진다.



0.8

제목이 잘린다



0.6 ✓

포스터는 보통 2 : 3 → 약 0.67

여기에 제목·장르 두 줄이 더 붙는다

→ 그래서 0.67보다 작은 값이 필요하다

글자가 잘리면? 값을 더 줄인다. (0.7 → 0.6)

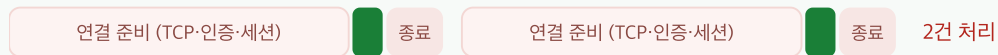
커넥션 풀 — 연결을 미리 뚫어 두고 빌려 쓴다

DB에 "연결하는 시간"이 정작 "쿼리 실행 시간"보다 훨씬 길다. 이 낭비를 없애는 게 커넥션 풀이다.

1 요청마다 새로 연결하면 무슨 일이 생이나

DB 연결 한 번은 TCP 연결 → 로그인 인증 → 세션 준비를 거친다. 여기에 수십 ms가 든다. 정작 쿼리는 1ms면 끝난다.

풀 없음 — 요청마다 새로 연결



커넥션 풀 — 미리 뚫어 두고 빌려 씀

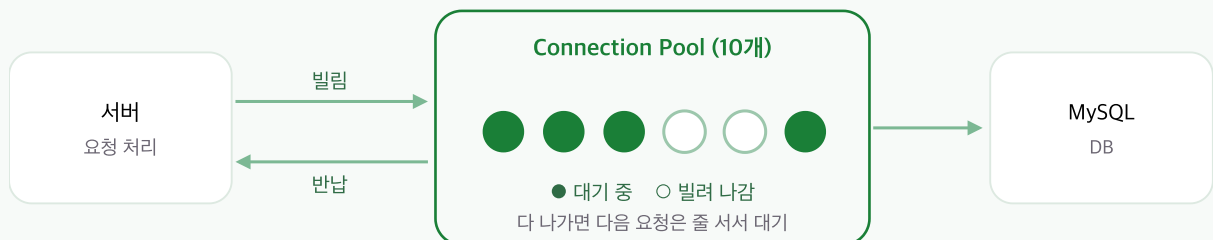


■ = 실제 쿼리(1ms). 나머지는 전부 연결을 준비하고 닫는 시간이다.

핵심 — 풀은 쿼리를 빠르게 만드는 게 아니다. **연결 준비 시간을 요청 경로에서 빼는 것이다.**

2 동작 방식

서버가 켜지면 연결을 몇 개 만들어 바구니에 담아 둔다. 요청이 오면 빌려 쓰고 반드시 돌려준다.



`connectionLimit 10 · waitForConnections true · queueLimit 0`

설정은 거의 없다

Spring이 대신 만들어 준다

풀을 직접 만들 일은 없다. 대신 언제 만들어지는지와 어떻게 관리되는지는 알아야 한다.

3 Spring은 자동이다

Spring Boot에는 HikariCP라는 풀이 이미 들어 있다. 아래 설정만 쓰면 풀이 저절로 만들어진다. (NestJS는 `mysql2`의 `createPool`을 직접 조립한다)

```
# application.yml - Spring은 이게 전부다
spring:
  datasource:
    url: jdbc:mysql://localhost:3306/book_rental
    username: ${DB_USER:root}
    password: ${DB_PASSWORD:}
```

우리가 쓴 `JdbcTemplate`은 풀에서 빌리고 → 쿼리하고 → 반납까지 알아서 한다. 그래서 Repository에서 연결을 직접 열 일이 없다.

4 기동 로그에 Hikari가 없다고 실패가 아니다

풀은 서버가 켜질 때가 아니라 첫 쿼리가 필요할 때 만들어진다(lazy 초기화).

서버 기동

Hikari 로그
아직 없음

GET /books

이때 풀 생성
HikariPool-1 Start completed

이후 요청

풀에서 빌리기만

조심할 것 둘, 그리고 오늘 한 장 요약

5 조심할 것 두 가지

커넥션 누수

빌린 연결을 돌려주지 않으면 바구니가 비고, 모든 요청이 줄만 서다가 서버 전체가 멈춘다.

최선의 방어는 직접 열고 닫지 않는 것 —

`JdbcTemplate`에 맡긴다.

풀은 클수록 좋은가 → 아니다

연결 하나마다 DB 쪽에도 스레드와 메모리가 붙는다. 서버를 여러 대 띄우면 DB의 `max_connections`를 넘겨 오히려 죽는다.

기본 10에서 시작해 측정하고 조정한다.

한 줄 정리 — 풀은 빌리고 반드시 반납하는 한정된 자원이다. 크기는 DB가 감당할 만큼만.

6 오늘 담당 파트 한 장 요약

FRONTEND

- 데이터는 한 곳에, 화면은 읽기만
- 없을 수 있으면 `Movie?` — ! 는 앱을 죽인다
- 사이에 뭘 넣으면 `separated`, 여러 열이면 `GridView`
- 가로 목록은 높이를 직접 준다
- `childAspectRatio`는 작을수록 세로로 길다

BACKEND

- 느린 건 쿼리가 아니라 연결 준비
- 풀 = 미리 뚫어 두고 빌리고 반납하는 바구니
- Spring은 HikariCP 자동, 설정은 datasource뿐
- 풀은 첫 쿼리 때 만들어진다(lazy)
- 반납 안 하면 서버가 멈춘다 · 크기는 DB가 감당할 만큼